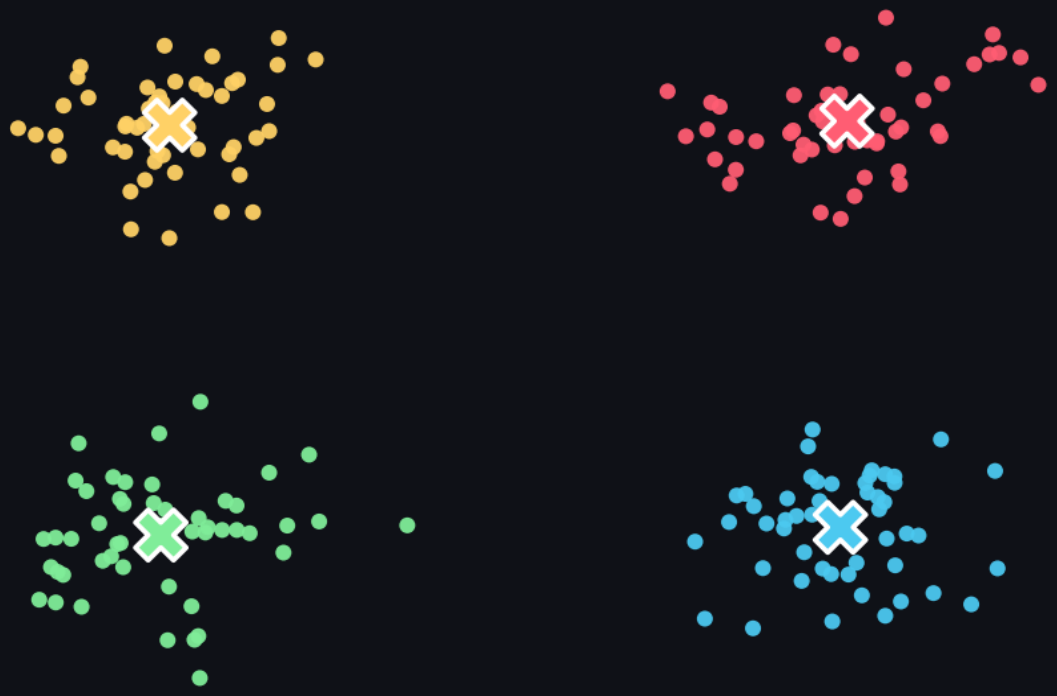
A VISUAL, STEP-BY-STEP TUTORIAL

K-Means Clustering

From Zero to Pro

Intuition • Maths • Elbow Method • scikit-learn • Code from scratch • Pro-level tricks



Built from three lecture scripts: **geometric intuition + elbow method**, **K-Means in Python (scikit-learn)**, and **K-Means from scratch**. Everything from the scripts is kept, tidied into a tutorial, and extended with maths, diagrams and pro-level material.

LEVEL 1 · BASIC	LEVEL 2 · CORE	LEVEL 3 · ELBOW
LEVEL 4 · PRACTICE	LEVEL 5 · BUILD	LEVEL 6 · PRO

Table of Contents

How to use this tutorial 4

Part 1: The Big Picture 5

1.1 Two families of machine learning 5
1.2 What is clustering? 5
1.3 What does the name “K-Means” mean? 6
1.4 Our running problem: the placement cell 6
1.5 “But I can see the groups with my eyes!” 7

Part 2: How K-Means Works 9

2.0 Vocabulary you need 9
2.1 The algorithm in five steps 9
2.2 Step 1: Decide K 10
2.3 Step 2: Initialise the centroids 10
2.4 Step 3: Assign every point to a cluster 11
2.5 Step 4: Move the centroids 12
2.6 Step 5: Are we finished? 13
2.7 Complete walkthrough, round by round 13
2.8 Why K-Means is so powerful 15

Part 3: Choosing K — the Elbow Method 16

3.1 What is WCSS? 16
3.2 Building the elbow curve step by step 17
3.3 WCSS always decreases as K grows 18
3.4 Finding the elbow point 19

Part 4: K-Means in Python with scikit-learn 21

4.1 Create (or load) the dataset 21
4.2 Look at the data first 21
4.3 Find K with the elbow method 21
4.4 Train the final model 22
4.5 Plot the clusters (boolean indexing) 23
4.6 Reading the clusters like a placement head 24
4.7 Useful things a trained model gives you 25
4.8 The same thing in 3D (and any dimension) 25

Part 5: Code K-Means from Scratch 27

5.1 Project set-up 27

5.2 The class skeleton . . . 27
5.3 Step 2: random centroids . . . 28
5.4 The main loop . . . 29
5.5 A general Euclidean distance (the dot-product trick) . . . 29
5.6 Writing assign_clusters . . . 30
5.7 Writing move_centroids . . . 31
5.8 The stopping check . . . 32
5.9 The complete class . . . 33
5.10 Plot the result . . . 33
5.11 Experiments: what does max_iter do? . . . 34
5.12 Test on the real student dataset . . . 35
Part 6: Pro-Level K-Means . . . 37
6.1 What K-Means really optimises . . . 37
6.2 Initialisation matters: n_init and k-means++ . . . 37
6.3 Feature scaling is not optional . . . 38
6.4 When K-Means fails (assumptions & limits) . . . 39
6.5 A second opinion on K: the silhouette score . . . 40
6.6 Speed: vectorise, and know the cost . . . 41
6.7 A real application: image compression . . . 41
6.8 scikit-learn KMeans: the parameters that matter . . . 42
6.9 Pro checklist . . . 42
Appendix A: One-page cheat sheet . . . 44
Appendix B: Glossary . . . 44
Appendix C: Notes on the original scripts . . . 44

How to use this tutorial

The tutorial is a ladder. Each level only needs the ones before it, so you can stop at any rung, or go all the way to the top. Whenever you see a coloured box, it has a special meaning:

Tip / Key idea
Green boxes hold the one thing you must remember from a section.

Note / Example
Blue boxes hold worked examples and side notes.

Maths corner
Yellow boxes hold the formulas. Every formula is explained in plain words right after it.

Analogy
Purple boxes explain an idea with an everyday picture.

Watch out
Red boxes are common mistakes and limits.

Try it yourself
Orange boxes are exercises and homework.

LEVEL 1 · BASIC

Part 1: The Big Picture

What is clustering? Why do we need K-Means? What problem are we solving?

Part 1: The Big Picture

1.1 Two families of machine learning

Before K-Means, one distinction matters: does the data come with the **answers** or not?

	Supervised learning	Unsupervised learning
Data looks like	Features + a known label (e.g. student → *placed / not placed*)	Features only. No label at all.
Goal	Learn to predict the label	Discover hidden structure
Examples	Spam filter, price prediction	**Clustering**, topic discovery

1.2 What is clustering?

Clustering means putting **similar things into one group**, and keeping different things in different groups. Nobody tells the algorithm what the groups are; it has to discover them from the data alone. That is why clustering is an **unsupervised** machine-learning technique.

Analogy: sorting laundry

You dump a pile of clothes on the bed. Nobody gave you labels, yet you naturally make piles: whites, darks, socks. You used *similarity* (colour, type) to make the groups. A clustering algorithm does the same with numbers.

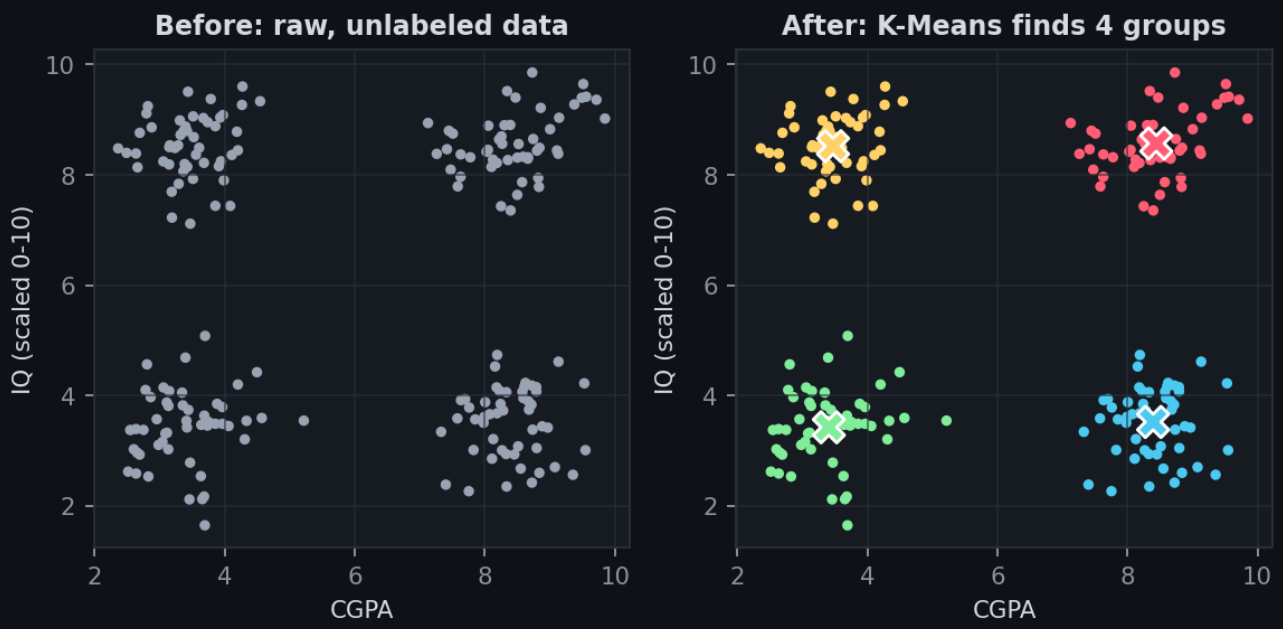


Figure 1: Unlabeled points (left) and the same points after clustering (right). The X markers are the cluster centres, called centroids.

Where is clustering used?

Domain	What gets clustered	Why it helps
Business	Customers	Create different offers / marketing for each customer group
Education	Students	Teach, support or prepare each group differently
Images	Pixels / colours / pictures	Compress images, segment objects (see Part 6)
Music / media	Songs	Group similar songs so you can recommend them

K-Means is one of the most famous and most widely used algorithms in all of machine learning. It is powerful, simple and fast, which is why it appears in almost every list of top ML algorithms.

1.3 What does the name “K-Means” mean?

- **K** = the number of clusters you want. If K = 3 you ask for 3 groups.
- **Means** = the centre of every group is the *mean* (average) of the points inside it.

So “K-Means” simply says: *“make K groups, and describe each group by the average of its members.”*

1.4 Our running problem: the placement cell

Imagine you work in a college's **placement cell**. You have data on the **final-year Computer Science students**. For each student you have two numbers: their **CGPA** and their **IQ**. You want to split the students into groups so that you can **prepare each group differently for placements** and improve the college's placement record.

Student	CGPA (0-10)	IQ (scaled 0-10)
Student 1	8.7	8.3
Student 2	3.9	3.9
Student 3	4.0	7.9
...	...	...
Student 200	...	...

> **About the numbers**
> The dataset is **generated** (a toy dataset), exactly like in the lectures: 200 students. To keep the story simple, both columns are written on a similar 0-10 scale (IQ is shown *rescaled*). Real datasets are messier, and in Part 6 you will see why different scales need care.

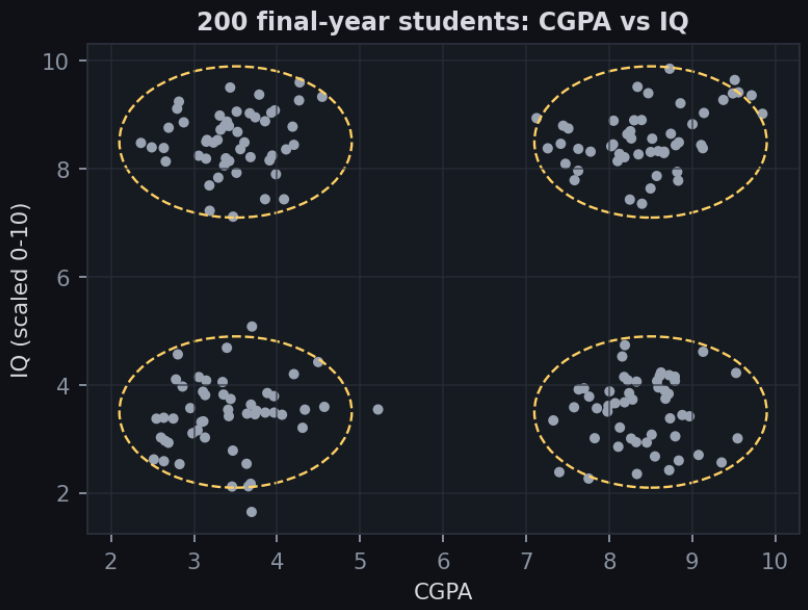


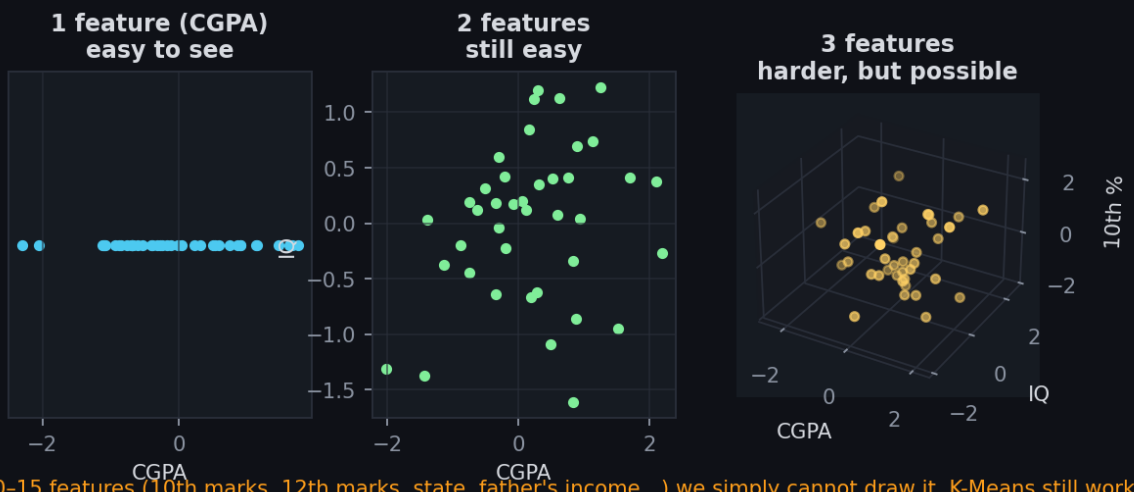
Figure 2: Our data. Even by eye you can imagine 4 blobs (dashed yellow circles).

1.5 “But I can see the groups with my eyes!”

Good question. Looking at Figure 2 you can already see roughly 3-4 groups, so why do we need an algorithm? Because **in 2D you do not need one**. Your eyes do the job. But real data rarely has two columns. What if we also had:

- marks in 10th grade,
- marks in 12th grade,
- which state the student comes from,
- the father's monthly income…

Now you have 10-15 columns, you **cannot draw the graph**, and your eyes are useless. This is exactly where K-Means shines.



…and with 10–15 features (10th marks, 12th marks, state, father's income…) we simply cannot draw it. K-Means still works.

Figure 3: We can draw 1, 2 or (barely) 3 features. Beyond that we are blind, but the algorithm is not.

> **Key idea**
> We will **learn K-Means in 2D** because we can draw it. But everything K-Means does in 2D, it does **in exactly the same way in 3D and in n dimensions**. That is the real power of this algorithm.

Check yourself (Part 1)

- Why is clustering called “unsupervised”? *(Answer: there are no labels/answers in the data.)*
- What does the K in K-Means stand for? *(The number of clusters.)*
- Why would we still need K-Means if we can eyeball 2D data? *(Real data has many columns; we cannot plot or eyeball it.)*

LEVEL 2 · CORE

Part 2: How K-Means Works

The 5 steps, the maths of distance and mean, and a full walkthrough

Part 2: How K-Means Works

2.0 Vocabulary you need

Word	Meaning
Cluster	A group of similar points.
K	How many clusters we ask for.
Centroid	The centre of a cluster = the mean of all its points. (K-Means has K centroids.)
Euclidean distance	Ordinary straight-line distance between two points.
Iteration	One full round of: assign points → move centroids.
Convergence	The moment when nothing changes any more, so the algorithm can stop.

2.1 The algorithm in five steps

K-Means works in **five steps**. Steps 3, 4 and 5 form a **loop** that repeats until the answer stops changing. Steps 3 and 4 are the backbone of the whole algorithm, so read them with extra care.

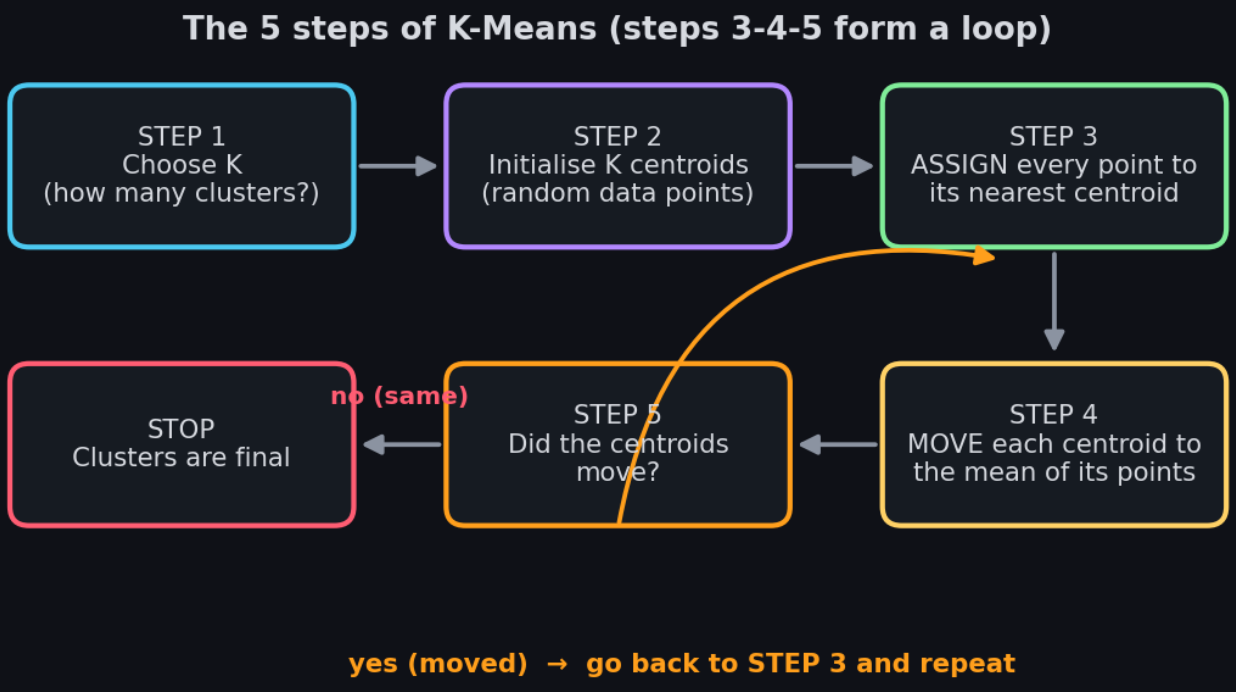


Figure 4: The complete K-Means flow-chart.

#	Step	One-line meaning
1	Decide K	Choose how many clusters to make.

#	Step	One-line meaning
2	Initialise centroids	Pick K random points as the first centres.
3	Assign clusters	Give every point to its nearest centroid.
4	Move centroids	Move each centroid to the mean of its points.
5	Check for finish	Stop if the centroids did not move; otherwise go to step 3.

2.2 Step 1: Decide K

This step surprises almost everyone the first time. The algorithm will do the clustering for you, **but you must tell it how many clusters to make**. By looking at the data, K-Means cannot decide whether there should be 3 clusters or 4. That is **your job** as the programmer or data scientist.

You may object: *"In high-dimensional data I cannot draw a graph, so how do I know how many clusters there should be?"* It is a tricky but interesting question, and it has a proper answer: the **Elbow Method** (Part 3). For now we **assume we already know K** and just study how the clustering happens. For our toy data we choose:

K = 3 (for the walkthrough in this part).

2.3 Step 2: Initialise the centroids

The algorithm needs a starting guess. If K = 3, it simply **picks any 3 data points at random** and declares them to be the 3 cluster centres (centroids). Let us call them C1, C2, C3.

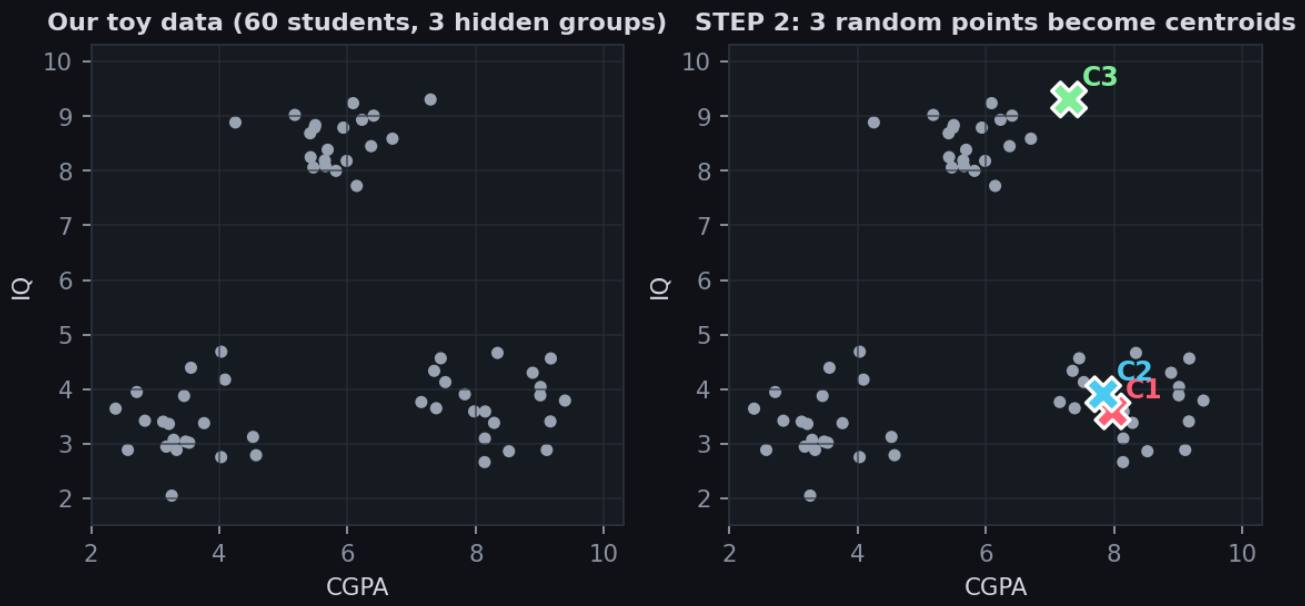


Figure 5: Left: the toy data (60 students). Right: three random data points become centroids. Notice that two of them landed in the same blob. That is a bad start, and K-Means will fix it.

> **Why is a bad start OK?**
>
> The start is only a guess. The next steps keep improving it. (Very unlucky starts can still cause trouble. Part 6 explains this and the smart fix called **k-means++**.)

2.4 Step 3: Assign every point to a cluster

Now the main work begins. For **every single point** we calculate its distance to **all K centroids**, and we give the point to the centroid that is **closest**. The distance we use is the **Euclidean distance**.

Maths corner: Euclidean distance

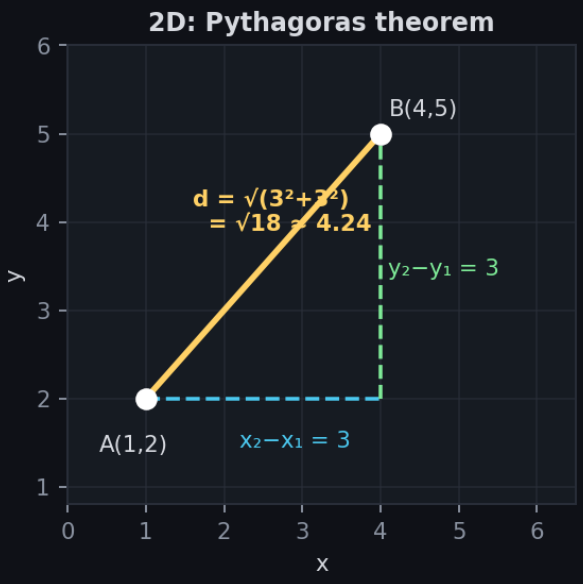
The straight-line distance between two points A(x_1, y_1) and B(x_2, y_2) comes from the Pythagoras theorem:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In plain words: take the difference in x, the difference in y, square both, add them and take the square root.

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}$$

The second line is the general version for **n dimensions**: a and b are lists of n numbers (n features), and Σ means “add up”. With 3 features there are 3 terms, with 10 features there are 10, and the idea never changes.



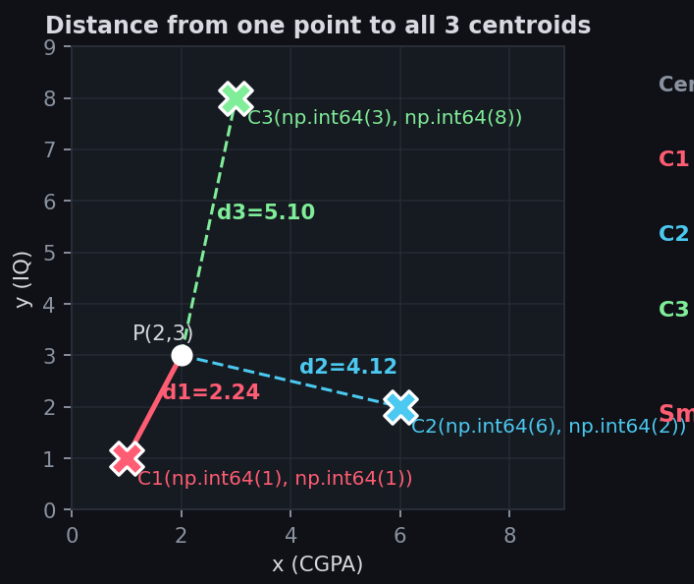
Dimensions	Formula
1D	$d = \lvert x_2 - x_1 \rvert$
2D	$d = \sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}$
3D	$d = \sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2 + (z_2-z_1)^2]}$
n-D	$d = \sqrt{[\ \Sigma\ (a_i - b_i)^2\ ]}$ for i = 1…n

Same idea in every dimension: square the differences, add them up, take the square root.

Figure 6: Euclidean distance is Pythagoras in every dimension.

Worked example: one point, three centroids

Take a student P = (2, 3) and three centroids C1 = (1, 1), C2 = (6, 2), C3 = (3, 8). We compute three distances and take the smallest:



Centroid	Calculation	Distance
C1 (1,1)	$\sqrt{(1^2+2^2)}=\sqrt{5}$	**2.24 ← minimum**
C2 (6,2)	$\sqrt{(4^2+1^2)}=\sqrt{17}$	4.12
C3 (3,8)	$\sqrt{(1^2+5^2)}=\sqrt{26}$	5.10

Smallest distance = C1 → P joins cluster 1 (red).

Figure 7: P is closest to C1 (distance 2.24), so P joins cluster 1.

Then we do the **exact same thing for every other point**. For instance, one point might be closest to the red centroid (it becomes red), another closest to blue (it becomes blue), and another closest to green. When all points are done, **step 3 is complete: every point has a cluster**.

In symbols

The cluster of point i is the index j of the centroid with the smallest distance:

$$c_i = \arg\min_{j \in \{1..K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|$$

“arg min” means “the j that makes this smallest”. μ_j is centroid number j.

2.5 Step 4: Move the centroids

We have just formed new clusters (a red group, a blue group, a green group), but the centroids are still at their old random places. A centroid is supposed to be the **centre of its cluster**, so we **move each centroid to the mean of its points**:

- the new x of the centroid = **mean of the CGPA** of all points in that cluster,
- the new y of the centroid = **mean of the IQ** of all points in that cluster.

Maths corner: the centroid (mean)

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$$

$|C_j|$ is the number of points in cluster j. Add all the points of the cluster (coordinate by coordinate) and divide by how many there are.

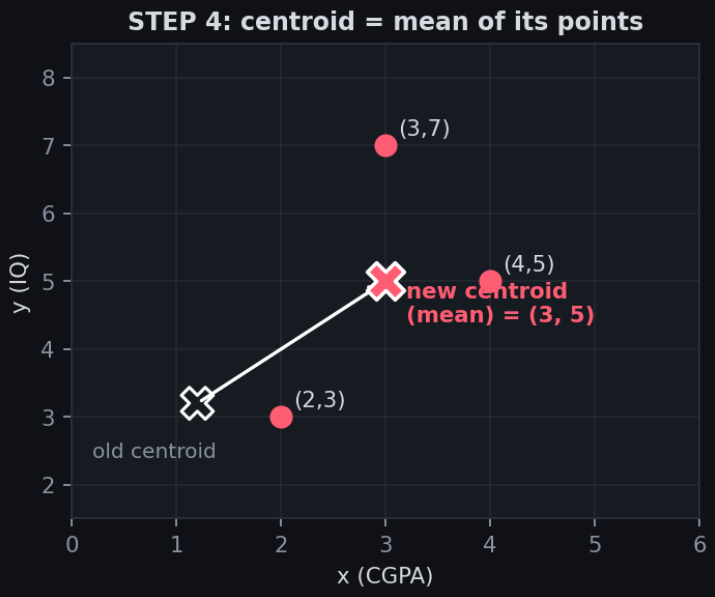


Figure 8: Cluster with points (2,3), (4,5), (3,7). Mean x = (2+4+3)/3 = 3, mean y = (3+5+7)/3 = 5. The centroid moves to (3,5).

Do this for the blue points (new blue centroid), the red points (new red centroid) and the green points (new green centroid). Every centroid typically **jumps** to a new position.

2.6 Step 5: Are we finished?

We must know when to stop. The logic is simple: **compare the current centroid positions with the positions from the previous step**.

- If the centroids **moved** (they are different), the clustering is still changing → **go back to step 3**.
- If the centroids **did not move** (they are the same), nothing will ever change again → **stop**. The clusters are final.

2.7 Complete walkthrough, round by round

Let us follow the algorithm on the toy data from Figure 5. Read Figure 9 from left to right, top to bottom. Small hollow X's with white arrows show where a centroid *was* and where it moved *to*.

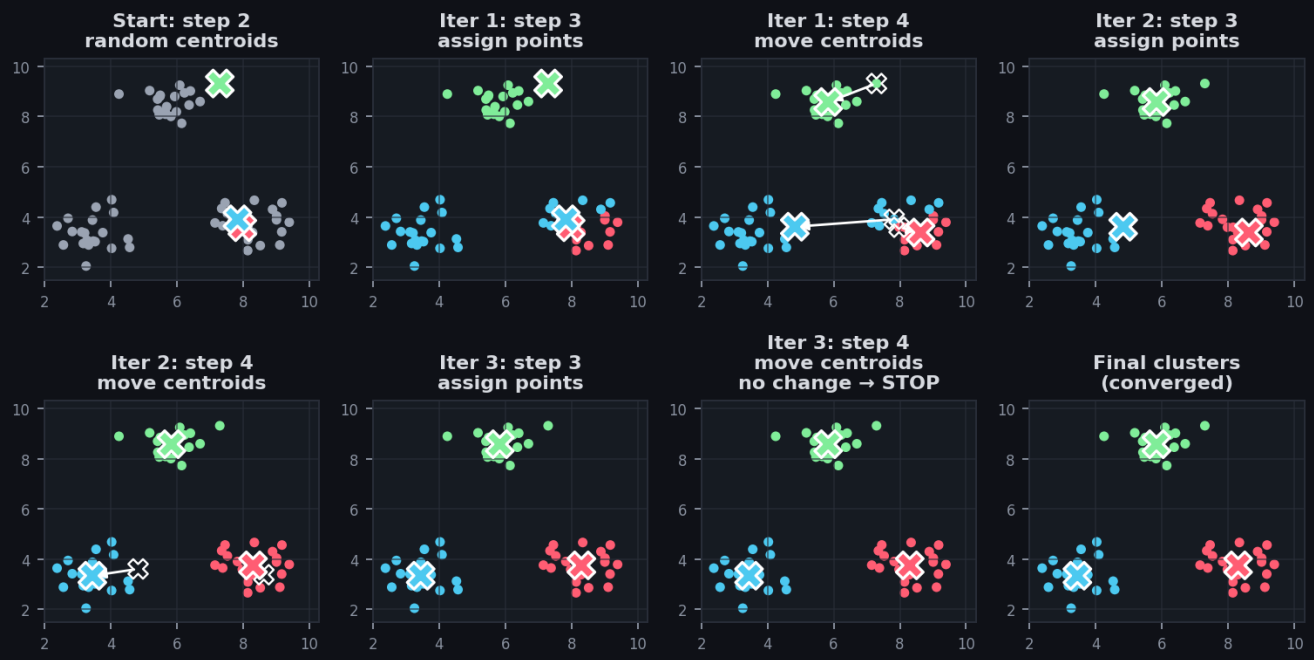


Figure 9: K-Means from random start to convergence.

What happened in each panel

- **Start (step 2):** three random points become centroids. Two of them (blue and red) sit in the same blob on the right; green sits in the top blob; nothing sits in the bottom-left blob. A poor start.
- **Iteration 1, step 3:** every point joins its nearest centroid. The bottom-left students are far from everything, so they all join the nearest one (blue, whose centroid is on the right).
- **Iteration 1, step 4:** centroids move to the means of their new groups. The blue centroid is dragged left (towards its many bottom-left members) and the red centroid moves to the right blob. Since the centroids moved, the algorithm must continue.
- **Iteration 2, step 3:** distances are recomputed with the new centroids. Some points are now **closer to a different centroid** and are reassigned: this is the key moment where clusters change. Points at the boundary swap to the nearer group.
- **Iteration 2, step 4:** centroids move again, a little less than before. They moved, so we continue.
- **Iteration 3, steps 3 and 4:** no point changes cluster, so every cluster has the same members, so the means are exactly the same, so **the centroids do not move**. Step 5 says: stop!
- **Final clusters:** three clean groups. (The clustering found the three hidden groups even though it started badly.)

> **Key idea: the two-move dance**
> K-Means repeats two simple moves: **(a)** given centroids, assign points; **(b)** given assignments, move centroids. Each move makes the clusters a little tighter, until nothing changes.

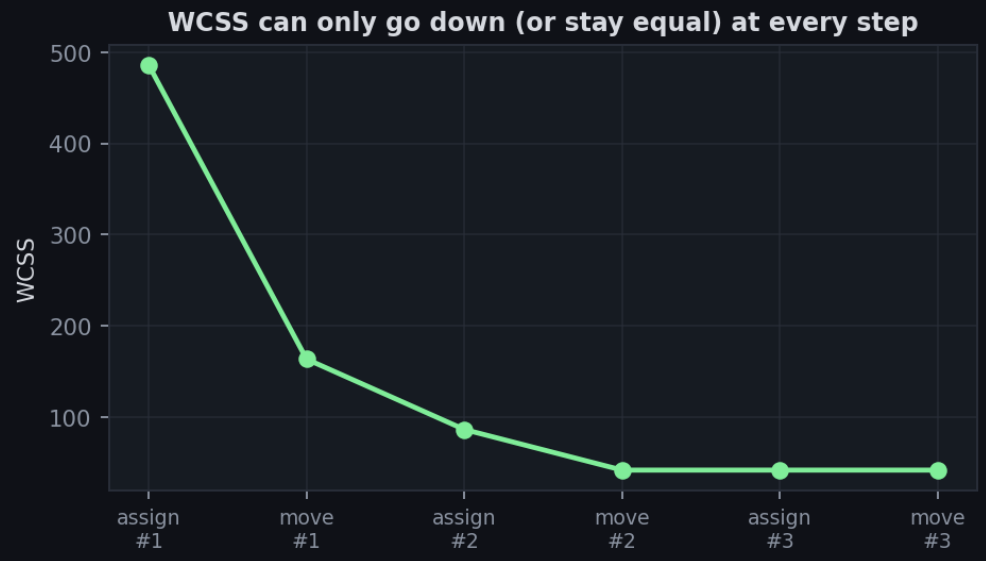


Figure 10: The total within-cluster squared distance (WCSS, explained in Part 3) never goes up during these steps. That is why K-Means always terminates.

2.8 Why K-Means is so powerful

To do the whole thing we used **only one geometric tool**: the distance between two points (plus the average). The great part is that distance works the same in 2D, 3D and n-D. So whatever you did here on a flat sheet of paper you can do on a dataset with hundreds of columns. **That is why K-Means is such a strong algorithm.**

> **Try it yourself: a dry run on paper**
> Take a notebook. Draw about 12 random points on a grid. Choose K = 2. Pick 2 of the points as centroids, measure distances with a ruler, colour the points, compute the means, move the centroids and repeat until nothing changes. Doing this dry run once makes the algorithm “stick” in your mind far better than reading it five times.

Check yourself (Part 2)

- Which steps repeat in the loop? *(3, 4 and 5: assign, move, check.)*
- A cluster has points (1,1), (3,1), (2,4). Where is its centroid? *(Mean $x = 2$, mean $y = 2 \rightarrow (2, 2)$.)*
- When does K-Means stop? *(When the centroids are in the same place as in the previous round.)*
- Who decides K? *(You do.)*

LEVEL 3 · ELBOW

Part 3: Choosing K with the Elbow Method

WCSS (inertia), the elbow curve, and why it works

Part 3: Choosing K — the Elbow Method

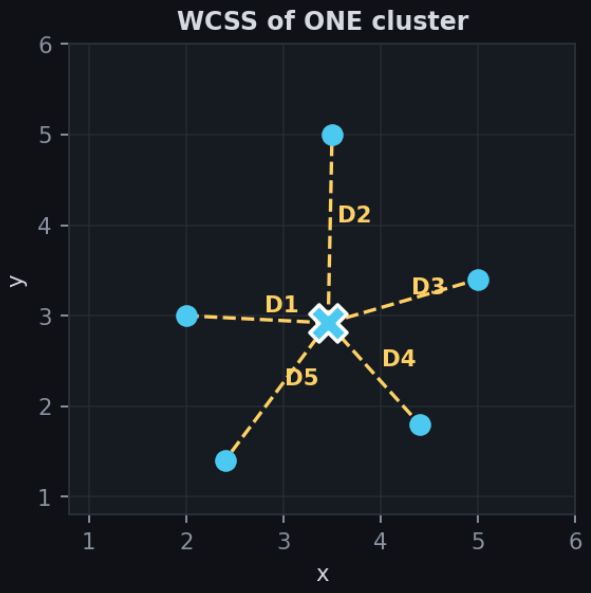
In Part 2 we *assumed* K. Now we answer the pending question: **how many clusters should a dataset be split into?** The most famous technique is the **Elbow Method**. You draw a graph called the **Elbow Curve**, which has two quantities:

- **X-axis:** the number of clusters (K).
- **Y-axis:** a quantity called **WCSS**, also known as **inertia**. WCSS stands for **Within-Cluster Sum of Squared distances**.

3.1 What is WCSS?

Imagine one cluster with its centroid. To get that cluster's WCSS:

- measure the distance from the centroid to **every point** in the cluster (call them D1, D2, D3, D4, D5…),
- **square** every distance,
- **add** all the squares together.



Step 1: measure D1…D5 (point → centroid)

Step 2: square each distance

Step 3: add them up

$WCSS = D1^2 + D2^2 + D3^2 + D4^2 + D5^2$

$= 2.14 + 4.33 + 2.60 + 2.14 + 3.43$

$= 14.64$

Small WCSS = points hug their centroid = tight cluster.

Figure 11: WCSS of one cluster. The five dashed lines are D1…D5.

Maths corner: WCSS

For a single cluster C_j with centroid μ_j:

$$\text{WCSS}(C_j) = \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$

For the **whole** clustering (several clusters) you simply add up the WCSS of every cluster:

$$\text{WCSS} = \sum_{j=1}^{K} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$

Read it from the inside out: for every point x in cluster j, take its squared distance to its centroid; add over all points in the cluster; then add over all K clusters.

Example with two clusters. If cluster A has WCSS = 12.4 and cluster B has WCSS = 7.1, then the total WCSS of the clustering is 12.4 + 7.1 = **19.5**.

What does WCSS tell us?

Small WCSS → points are packed tightly around their centroids → compact, good clusters. **Large WCSS** → points are spread out far from their centroids.

3.2 Building the elbow curve step by step

We try K = 1, then 2, then 3, and so on, and for each K we run K-Means and record the total WCSS:

- **K = 1:** all data is one big cluster, with its centroid somewhere in the middle. Compute its WCSS and plot the point.
- **K = 2:** two clusters, two centroids. Compute the WCSS of each, add them, plot the sum at K = 2.
- **K = 3:** three clusters. Compute three WCSS values, add them, plot at K = 3.
- …and continue for K = 4, 5, … 10 (or even 15 or 20).



Figure 12: The same data clustered with K = 1 to 6. The WCSS is printed above each panel. Notice the huge drops up to K = 3 and the tiny drops after that.

Why not try K = 20,000? Because making 20 or 20,000 clusters out of a small dataset is illogical: the clusters become meaningless and there is no benefit left from clustering. So we only try a sensible range, then look at the graph.

3.3 WCSS always decreases as K grows

Let $WCSS_1$ be the WCSS with one cluster, $WCSS_2$ with two clusters, $WCSS_3$ with three, and so on. Is it true that **$WCSS_1$ is the biggest, $WCSS_2$ is smaller, $WCSS_3$ is smaller still**, always going down? **Yes.** Here is the logical proof.

Proof by extreme case

Suppose there are **50 points**. The maximum number of clusters you can make is 50: every point becomes its own cluster, and therefore **its own centroid**. Then the distance from each point to its centroid is **0**. The WCSS of each cluster is 0, and 0 + 0 + … + 0 (50 times) = **0**.

So at K = 1 the WCSS is as large as it can ever be (one centroid far from many points), and at K = n it is exactly zero. In between, adding clusters only lets points get closer to a centroid, never farther, so the curve keeps going down.

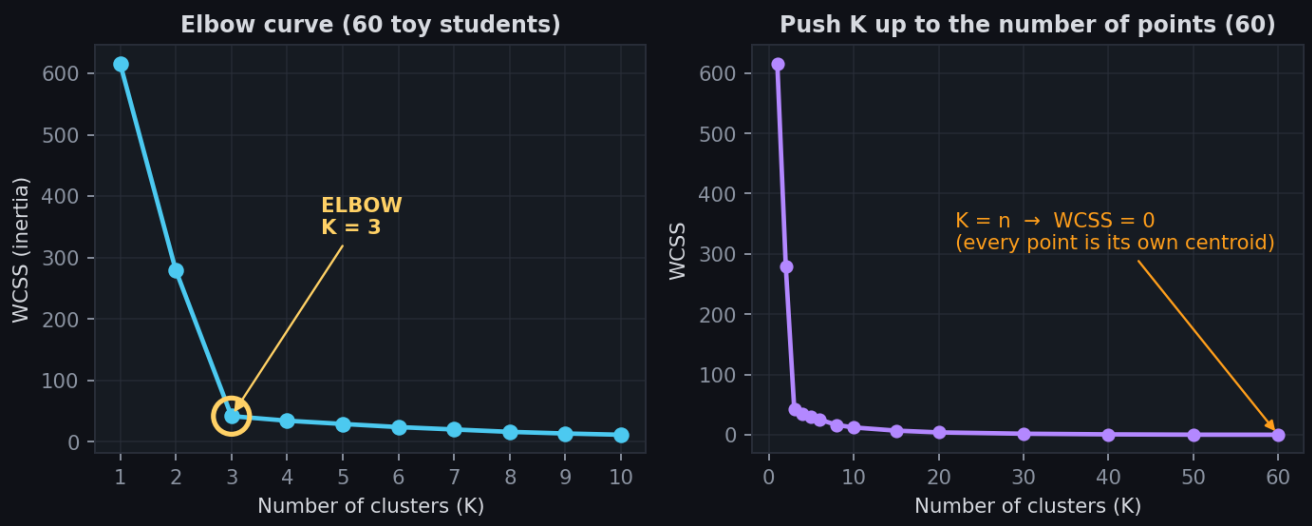


Figure 13: Left: the elbow curve for our toy data. Right: pushing K all the way to the number of points (60) drives WCSS to zero.

3.4 Finding the elbow point

The curve falls steeply and then flattens. The **elbow point** is exactly where the big drop in WCSS **starts to stabilise**. In Figure 13 (left), the curve is steep until K = 3 and then flattens, so the elbow is at **K = 3**, which matches the three hidden groups we know are in the data.

> **The mountain analogy**
> Imagine the curve is a mountain and you are **falling from the top**. At the beginning you fall very fast and feel very scared. As you come down, the slope is less steep and you feel a little less scared. The point where your **fear starts to drop** (“I won't get hurt now”) is the **elbow point**.

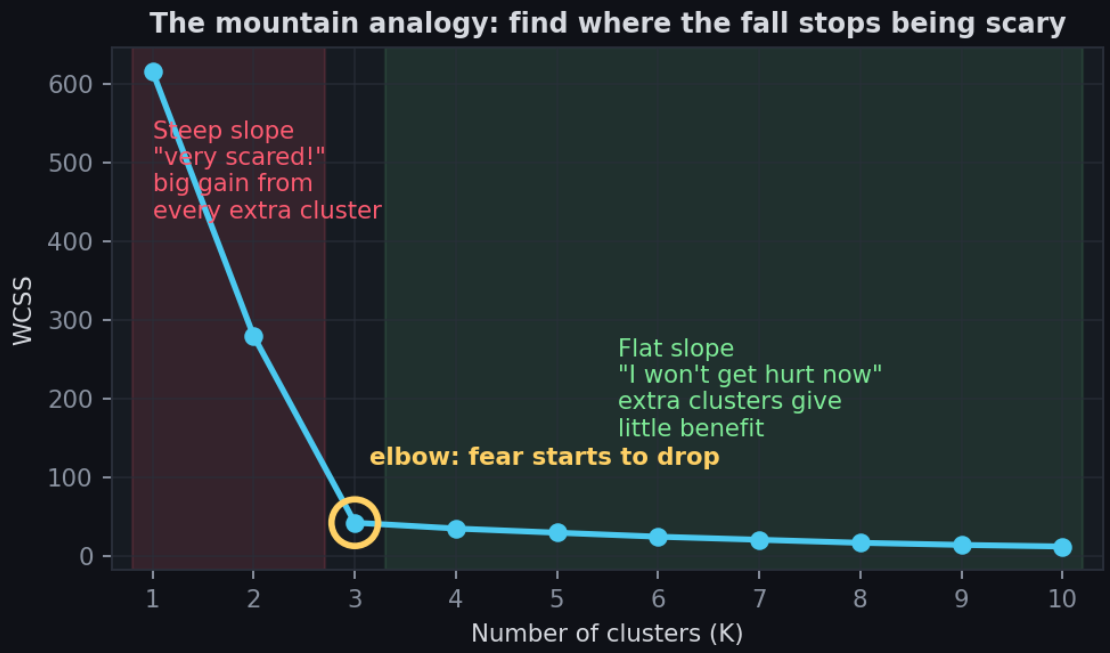


Figure 14: The mountain analogy on the elbow curve.

The maths intuition behind the elbow

K	WCSS	Drop from K−1	% of the first WCSS removed
1	615.6	-	-
2	279.0	336.6	54.7%
3	41.8	237.3	38.5%
4	34.4	7.4	1.2%
5	29.1	5.3	0.9%
6	24.0	5.1	0.8%
7	20.1	3.9	0.6%
8	16.2	3.9	0.6%

Read the table like a shopkeeper counting benefit per rupee spent. Going from **1 to 2** clusters removes a huge chunk of WCSS: very beneficial. Going from **2 to 3** is also very beneficial. But going from **3 to 4** barely helps, and 4 → 5 helps even less. Extra clusters beyond that point do not yield much benefit, so we **catch the point where the significant benefit stops**. Here that point is **K = 3**.

> **Elbow method limits**
> Real curves are not always so clean. Sometimes the bend is gentle and two people pick different K. The elbow is a *guide*, not a law. In Part 6 you will meet a second opinion: the **silhouette score**. Domain knowledge (what K makes business sense?) also matters.

Check yourself (Part 3)

- What does WCSS measure? *(Sum of squared distances from each point to its own cluster's centroid.)*
- What is WCSS when K = number of points? *(Zero.)*
- Why is the elbow the “best” K? *(Before it, each extra cluster helps a lot; after it, extra clusters add little.)*

LEVEL 4 · PRACTICE

Part 4: K-Means in Python (scikit-learn)

A real run: elbow, fit_predict, plotting, interpretation, and the 3D case

Part 4: K-Means in Python with scikit-learn

Time to code. The data is the 200-student dataset: the final-year students of a college with **CGPA** and **IQ**. We want to cluster them using these two values. The data is generated, so the clusters are very clear; in the real world you will not get such tidy blobs.

4.1 Create (or load) the dataset

If you have the file *student_clustering.csv* just load it with **pd.read_csv**. If not, this snippet generates an equivalent file so you can follow along:

Python · load the data

```python
import numpy as np
import pandas as pd

rng = np.random.RandomState(7)
centers = [(8.5, 8.5), (8.5, 3.5), (3.5, 8.5), (3.5, 3.5)]   # (CGPA, IQ-scaled)
X = np.vstack([np.c_[rng.normal(c, 0.6, 50), rng.normal(i, 0.6, 50)]
               for c, i in centers])
df = pd.DataFrame(X, columns=["cgpa", "iq"]).sample(frac=1, random_state=0)
df.to_csv("student_clustering.csv", index=False)

df = pd.read_csv("student_clustering.csv")
df.head()
```

4.2 Look at the data first

Python · scatter plot

```python
import matplotlib.pyplot as plt

plt.scatter(df["cgpa"], df["iq"])
plt.xlabel("CGPA"); plt.ylabel("IQ")
plt.show()
```

We see 4 well separated blobs (it is a toy dataset, so they are crisp). We will feed exactly this data to the algorithm and see whether it clusters it properly.

4.3 Find K with the elbow method

Clearly 4 clusters are visible, but let us **check it with the elbow method**, as you would for data you cannot see. Import KMeans, create an empty list called **wcss**, run a loop from 1 to 10, fit a KMeans with **n_clusters = i** on the whole dataframe, and append the **inertia_** attribute (this is WCSS):

Python · elbow method

```python
from sklearn.cluster import KMeans

wcss = []
for i in range(1, 11):                      # K = 1 ... 10
    km = KMeans(n_clusters=i, n_init=10, random_state=0)
    km.fit(df)                              # the whole dataframe is the input
    wcss.append(km.inertia_)                # inertia_ = WCSS

plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of clusters (K)"); plt.ylabel("WCSS")
plt.show()
```

The WCSS starts from a very big value (about **2664** for our data; the exact number depends on the dataset) and decreases very rapidly. After K = 4 the rate of decrease suddenly drops: **K = 4 is the elbow point**.

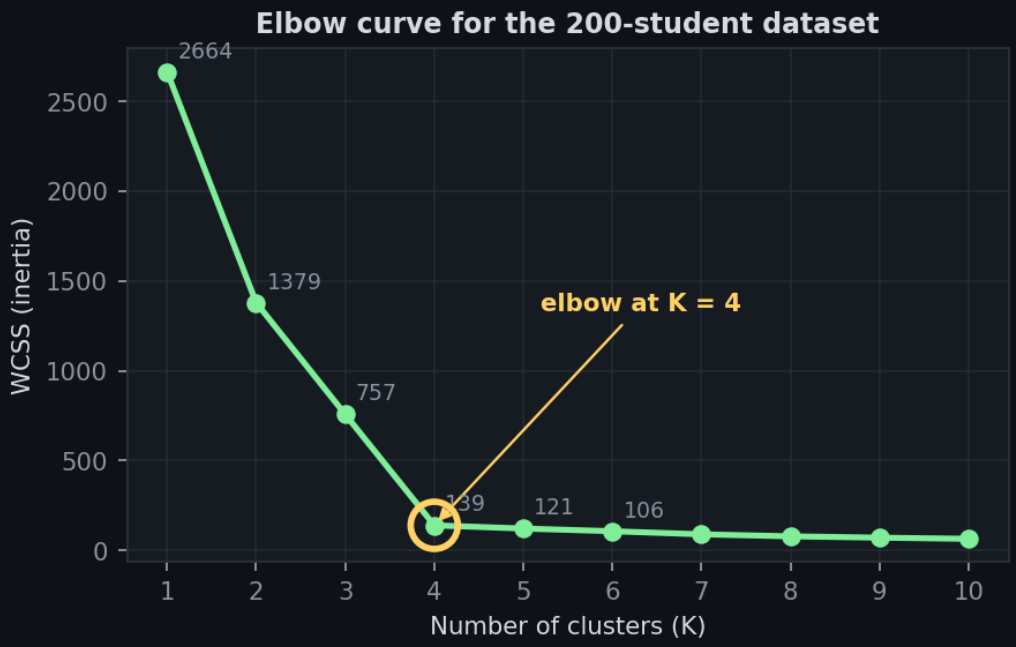


Figure 15: Elbow curve of the 200 students. The curve bends sharply at K = 4.

Notes on the code

n_init=10, random_state=0 are good habits: K-Means starts from random centroids, so we run it 10 times and keep the best, and we fix the random seed so results are repeatable (Part 6 explains why). In recent scikit-learn versions n_init defaults to “auto”, so older tutorials without it still run.

4.4 Train the final model

Now create a model with **n_clusters = 4**. First extract the numbers into a NumPy array with **.values** (easier to handle), then call **fit_predict**. *fit_predict* trains the model **and** returns the cluster number of every row.

Python · final model

```
X = df.values                                   # DataFrame -> NumPy array
km = KMeans(n_clusters=4, n_init=10, random_state=0)
y_means = km.fit_predict(X)
y_means[:10]
```

OUTPUT

```
array([0, 1, 2, 3, 1, 1, 0, 2, 0, 1], dtype=int32)   # one cluster number (0-3) per student
```

For the first row it is one cluster number, for the next row another, and so on. We call this array **y_means**. Note that the numbers 0-3 are just *names*: which group is called 0 can change from run to run.

4.5 Plot the clusters (boolean indexing)

To draw each cluster in its own colour we use **boolean indexing**. The expression **y_means == 0** gives True for every row in cluster 0. So **X[y_means == 0]** keeps only the rows of cluster 0, and **X[y_means == 0, 0]** picks their **first column** (x = CGPA) while **X[y_means == 0, 1]** picks their **second column** (y = IQ).

Python · colour each cluster

```
plt.scatter(X[y_means == 0, 0], X[y_means == 0, 1], color="blue")
plt.scatter(X[y_means == 1, 0], X[y_means == 1, 1], color="red")
plt.scatter(X[y_means == 2, 0], X[y_means == 2, 1], color="green")
plt.scatter(X[y_means == 3, 0], X[y_means == 3, 1], color="yellow")
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1],
            marker="X", s=200, color="white", edgecolor="black")   # the centroids
plt.show()
```

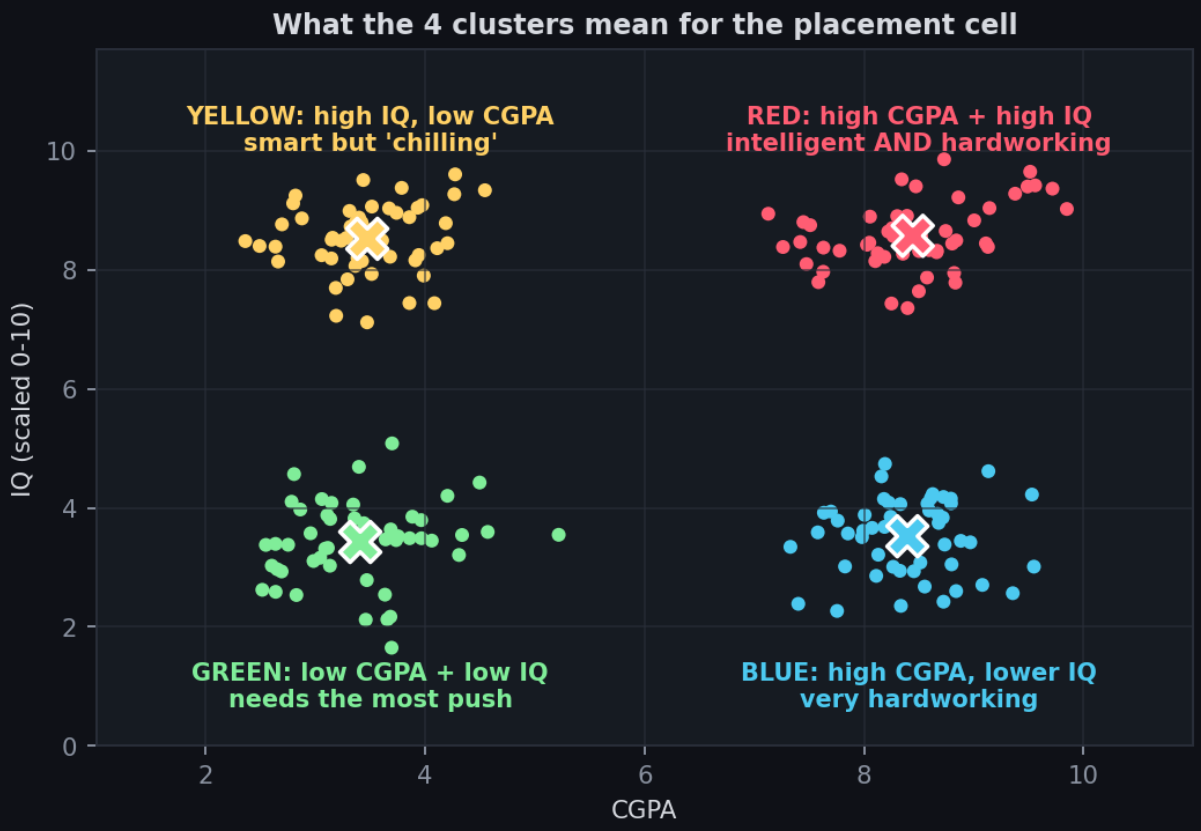


Figure 16: The four clusters of students and what each one means. (Cluster numbers and colours are arbitrary labels; we colour by meaning here.)

4.6 Reading the clusters like a placement head

Clustering is only useful when you **interpret** the groups. Look at the position of each group on the CGPA/IQ plane:

Group	CGPA	IQ	Who are they?	What should the placement cell do?
Red	High	High	Intelligent *and* hardworking	No need to push much; they are doing fine.
Blue	High	Lower	Not extremely high IQ, but very hardworking	Push them more; their effort deserves support.
Yellow	Low	High	Smart, but “chilling” in life	Motivate them: the talent is there, the effort is not.
Green	Low	Low	Neither CGPA nor IQ is high	They need the hardest work and the most pushing forward.

So you can **plan a different approach for every group** according to your data. In exactly the same way you can cluster **customers** (different marketing for each group), **songs** (to recommend similar ones) and **images** (to compress or segment them).

4.7 Useful things a trained model gives you

Attribute / method	What it gives you
`km.cluster_centers_`	The K centroid coordinates (shape K × number of features).
`km.labels_`	Cluster number of every training row (same as fit_predict's output).
`km.inertia_`	The final WCSS.
`km.n_iter_`	How many assign-and-move rounds it needed before stopping.
`km.predict([[8.0, 4.0]])`	Which cluster a **new** student belongs to (nearest centroid).

4.8 The same thing in 3D (and any dimension)

Most YouTube demos show 2D only. Here is the 3D look-and-feel. The dataset has **three columns** instead of two, generated with **make_blobs** (you choose the number of features and centres). The code is **identical** to the 2D one: elbow loop, then fit_predict with K = 4.

Python · 3D example with Plotly

```python
import pandas as pd, plotly.express as px
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, y = make_blobs(n_samples=300, n_features=3, centers=4, random_state=11)

# elbow curve (identical code to 2D)
wcss = [KMeans(n_clusters=i, n_init=10, random_state=0).fit(X).inertia_
        for i in range(1, 11)]

km = KMeans(n_clusters=4, n_init=10, random_state=0)
y_means = km.fit_predict(X)

df3 = pd.DataFrame({"x_1": X[:, 0], "x_2": X[:, 1], "x_3": X[:, 2],
                    "cluster": y_means.astype(str)})
fig = px.scatter_3d(df3, x="x_1", y="x_2", z="x_3", color="cluster")
fig.show()
```

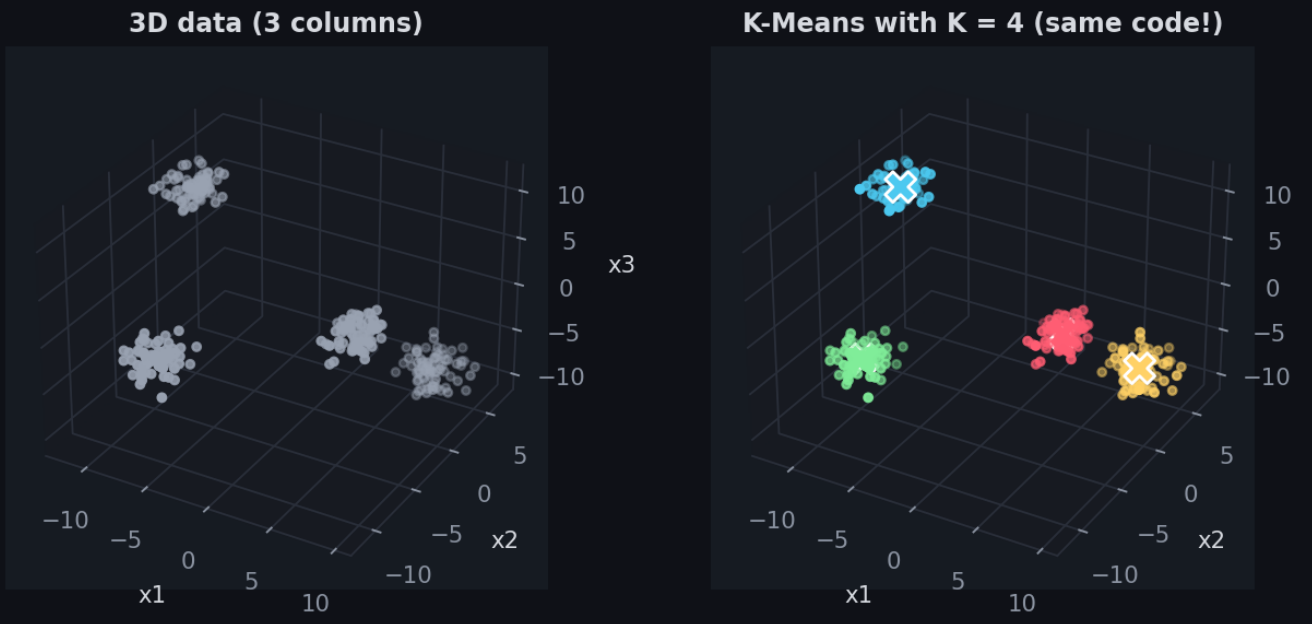


Figure 17: 3D data before and after K-Means (drawn here with matplotlib; Plotly gives you an interactive version you can rotate).

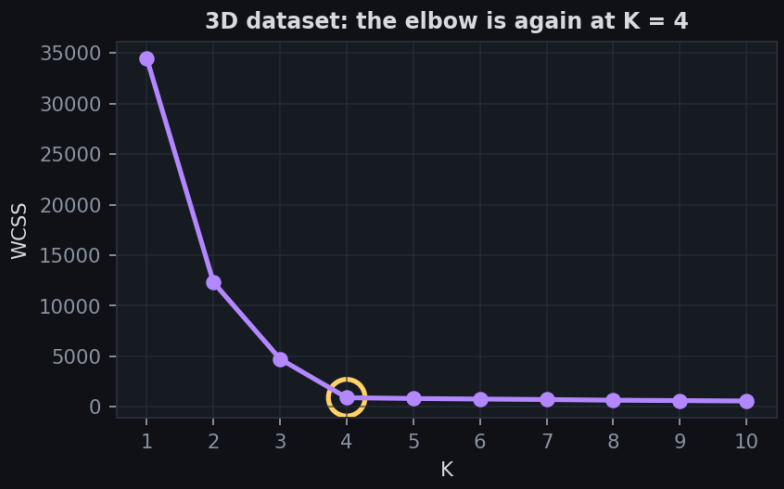


Figure 18: The elbow is again at K = 4.

Key idea

The **same piece of code works in any dimension**, because K-Means only needs distances and averages. This remains true in high dimensions where you can no longer draw anything.

Try it yourself

Generate a 3-feature dataset with 5 centres, find K with the elbow, and plot it. Then try 5 features: you cannot plot it, but you can still print **km.cluster_centers_** and see five numbers per centroid.

LEVEL 5 · BUILD

Part 5: Code K-Means from Scratch

Your own KMeans class with NumPy: assign, move, stop

Part 5: Code K-Means from Scratch

If you want to master a machine-learning algorithm, the best way is to **code it from scratch**. If you can code it yourself, you truly understand it. K-Means is a lovely candidate: short, visual and fun. We will build a class called **KMeans** with the same shape as scikit-learn's. I recommend reading Part 2 once more first, because we now translate its five steps into code.

5.1 Project set-up

Create two files:

- **app.py**: where the dataset lives (you can also load an external dataset) and where we test our class.
- **kmeans.py**: empty for now. Here we write the KMeans class.

The test dataset is simple: two blobs made with **make_blobs**, where we give the cluster centres and standard deviations.

app.py · the dataset

```
# app.py
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

centers = [(-5, -5), (5, 5)]
cluster_std = [2, 2]

X, y = make_blobs(n_samples=100, n_features=2, centers=centers,
                  cluster_std=cluster_std, random_state=42)

plt.scatter(X[:, 0], X[:, 1])
plt.show()
```

5.2 The class skeleton

The constructor receives two inputs. **n_clusters**: how many clusters (default 2). **max_iter**: the maximum number of times we repeat the loop “assign clusters, move centroids” (default 100). We also create a variable **centroids**, initially **None**, because we do not know the centroids yet.

kmeans.py · step 1: skeleton

```python
# kmeans.py
class KMeans:

    def __init__(self, n_clusters=2, max_iter=100):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.centroids = None

    def fit_predict(self, X):
        print(X)          # for now, just check that the data arrives
```

In app.py, import the class, create an object (**n_clusters = 2, max_iter = 100**) and call **fit_predict(X)**. For now it should just print X: a 2D array with 100 rows, each row holding an x and a y coordinate. Everything works so far.

app.py

```python
from kmeans import KMeans

km = KMeans(n_clusters=2, max_iter=100)
y_means = km.fit_predict(X)
```

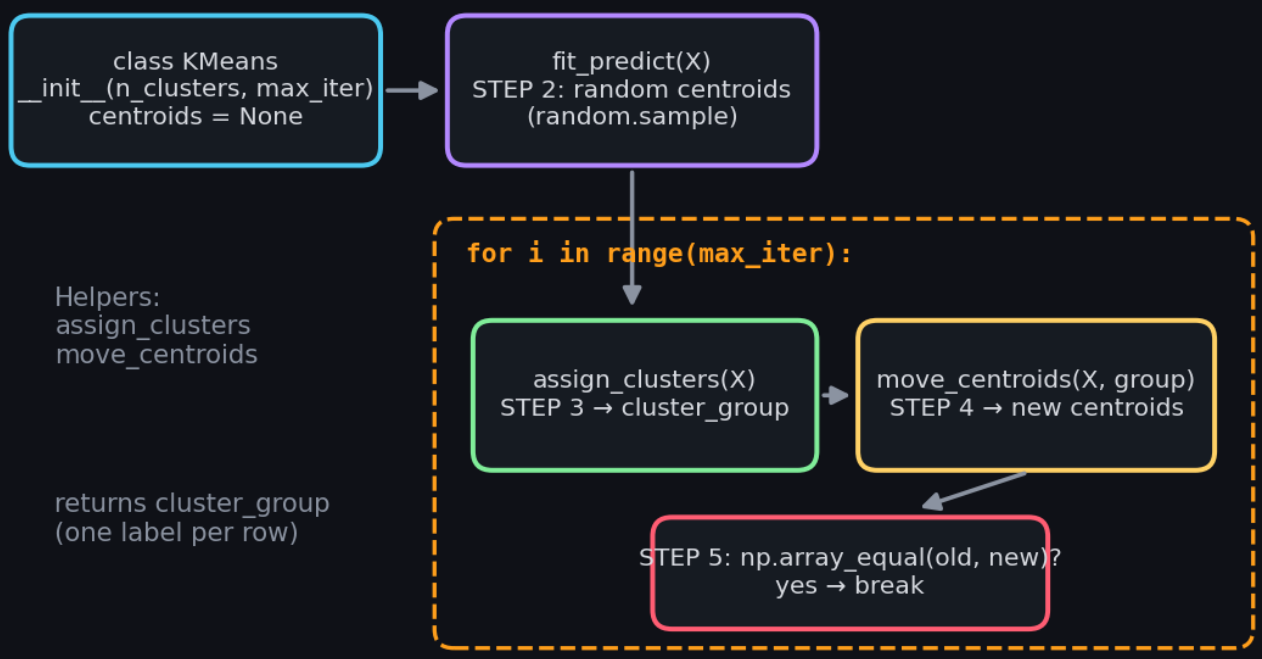


Figure 19: The structure of the class we are building.

5.3 Step 2: random centroids

Following the algorithm: step 1 (choose K) is done by the user. Step 2: select random centroids. X has 100 points; from these we randomly select **as many points as n_clusters**. We use **random.sample**. It needs two things: what to sample from (the row numbers **range(0, X.shape[0])**) and how many numbers to draw (**self.n_clusters**).

kmeans.py · step 2

```
import random

class KMeans:
    ...
    def fit_predict(self, X):
        random_index = random.sample(range(0, X.shape[0]), self.n_clusters)
        self.centroids = X[random_index]
        print(self.centroids)
```

OUTPUT

```
[[ 4.8  5.3]
 [-5.3 -5.1]]      # different rows every time you run it
```

Run it several times: every run selects two other random rows. **Step 2 is done.**

5.4 The main loop

Steps 3, 4 and 5 repeat, but we do not know how many times. So we use a loop that runs **at most max_iter** times, and inside it we do three tasks: **assign clusters**, **move centroids** and **check whether to finish**.

inside fit_predict

```
for i in range(self.max_iter):
    # assign clusters
    cluster_group = self.assign_clusters(X)
    # move centroids
    old_centroids = self.centroids
    self.centroids = self.move_centroids(X, cluster_group)
    # check finish
    if np.array_equal(old_centroids, self.centroids):
        break

return cluster_group
```

The function **assign_clusters** returns a **cluster_group**: one cluster number for every row. Suppose we had 5 rows; it might return **[0, 1, 0, 0, 1]**, meaning rows 1, 3 and 4 went to cluster 0 and rows 2 and 5 to cluster 1.

5.5 A general Euclidean distance (the dot-product trick)

In **assign_clusters** we need the distance between a row and a centroid. The textbook formula for two points (x1, y1) and (x2, y2) is $\sqrt{(x2-x1)^2 + (y2-y1)^2}$. The problem: if a third dimension z appears you must add $(z2-z1)^2$, and for an n-th dimension an n-th term. **The formula keeps changing as dimensions grow**, so we cannot hard-code it. We want one expression that works in any dimension. Look carefully:

Maths corner: from the formula to a dot product

Forget the square root for a moment. Then $(x_2-x_1)^2 + (y_2-y_1)^2$ is $(x_2-x_1)\cdot(x_2-x_1) + (y_2-y_1)\cdot(y_2-y_1)$. That is the **dot product** of the vector $[x_2-x_1, y_2-y_1]$ with **itself**. If we call the two points a and b, this vector is just **b − a**:

$$(x_2 - x_1)^2 + (y_2 - y_1)^2 = (\mathbf{b} - \mathbf{a}) \cdot (\mathbf{b} - \mathbf{a})$$

So the distance in any number of dimensions is:

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{(\mathbf{a} - \mathbf{b}) \cdot (\mathbf{a} - \mathbf{b})}$$

In NumPy: **np.sqrt(np.dot(a - b, a - b))**. If a z-term exists, the subtraction a − b automatically contains it, and you change **nothing**.

Check it with numbers. Let a = [1, 2] and b = [4, 5]. Then 4−1 = 3, 3^2 = 9, 5−2 = 3, 3^2 = 9, 9 + 9 = 18, and √18 = **4.24**. Now add a third dimension: a = [1, 2, 3], b = [4, 5, 6]. The sum becomes 9 + 9 + 9 = 27 and √27 = **5.196**. The code is the same for both:

Python · distance in any dimension

```python
import numpy as np

a = np.array([1, 2]);       b = np.array([4, 5])
np.sqrt(np.dot(a - b, a - b))              # 4.2426...  (= sqrt(18))

a = np.array([1, 2, 3]);    b = np.array([4, 5, 6])
np.sqrt(np.dot(a - b, a - b))              # 5.1961...  (= sqrt(27))
```

5.6 Writing assign_clusters

We must compute the distance of **every point with every centroid**. With 100 points and 2 centroids that is 100 × 2 = **200 distances**. Then for each point we ask: which centroid is the nearest? If it is the 0th centroid, the point gets label 0; if it is the 1st, label 1.

The plan, using a **nested loop**: the outer loop takes the 100 rows; the inner loop goes over the centroids (2 times), so the nested loop runs 200 times. For every row we collect its distances in a list, find the smallest with **min**, find *where* it sits with **index**, and append that position to **cluster_group**.

kmeans.py · assign_clusters

```python
def assign_clusters(self, X):
    cluster_group = []
    distances = []

    for row in X:
        for centroid in self.centroids:
            distances.append(np.sqrt(np.dot(row - centroid, row - centroid)))
        min_distance = min(distances)
        index_position = distances.index(min_distance)
        cluster_group.append(index_position)
        distances.clear()          # IMPORTANT: start fresh for the next row

    return np.array(cluster_group)
```

A bug worth remembering

Without **distances.clear()** the list keeps growing, so for row 2 you would be searching among 4 values (2 old + 2 new) and the index would be wrong. Always reset per-row temporary lists. We also return a NumPy array, because we need boolean comparisons on it in the next step.

Sanity test: print **cluster_group**. It should be an array of exactly 100 items (one per row) containing only 0s and 1s. If so, this function works.

5.7 Writing move_centroids

The cluster_group already says which points are in cluster 0 and which are in cluster 1. So we only separate the points of each cluster and take their **mean**. First find out how many unique clusters exist with **np.unique** (tomorrow someone may use our class with more than 2 clusters). Then loop over each cluster type, keep the rows of X whose label equals that type, and compute the mean with **axis=0**, which means “down the rows, once per column”, giving a separate mean for x and for y.

X	cluster_group	X[cluster_group == 0]	X[cluster_group == 1]
[1, 2]	1		[1, 2]
[2, 3]	0	[2, 3]	
[3, 4]	0	[3, 4]	
[5, 6]	1		[5, 6]
		rows where group is 0	rows where group is 1
		.mean(axis=0) = [2.5, 3.5]	**.mean(axis=0) = [3.0, 4.0]**

Figure 20: Boolean indexing + mean(axis=0) on a tiny example.

Python · the idea on a tiny array

```
X = np.array([[1, 2], [2, 3], [3, 4], [5, 6]])
cluster_group = np.array([1, 0, 0, 1])

np.unique(cluster_group)                # array([0, 1])
X[cluster_group == 0]                   # [[2, 3], [3, 4]]
X[cluster_group == 0].mean(axis=0)      # [2.5, 3.5]
X[cluster_group == 1].mean(axis=0)      # [3. , 4. ]
```

axis=0 vs axis=1

mean() alone averages *every* number into one value (not what we want). **axis=0** averages each *column* separately: x-mean and y-mean. **axis=1** would average each *row*: wrong here. If you get a wrong shape, check your axis.

kmeans.py · move_centroids

```
def move_centroids(self, X, cluster_group):
    new_centroids = []

    cluster_type = np.unique(cluster_group)

    for type in cluster_type:
        new_centroids.append(X[cluster_group == type].mean(axis=0))

    return np.array(new_centroids)
```

5.8 The stopping check

Back in **fit_predict**: before moving the centroids, save the current ones in **old_centroids**. After moving, compare with **np.array_equal(old_centroids, self.centroids)**. If they are equal, **break** the loop; if not, the loop keeps running (at most max_iter times). Finally return the last **cluster_group**.

> **Why does old_centroids keep the old values?**
> **old_centroids = self.centroids** does not copy the numbers: it just gives the same array a second name. It works here because **move_centroids** returns a *brand-new* array and we then re-point **self.centroids** to it, so old_centroids still points at the old array. If you ever modify the centroid array *in place*, use **.copy()**.

5.9 The complete class

kmeans.py · full listing (tested)

```python
# kmeans.py  (complete)
import random
import numpy as np


class KMeans:

    def __init__(self, n_clusters=2, max_iter=100):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.centroids = None

    def fit_predict(self, X):
        # STEP 2: pick n_clusters random rows of X as the first centroids
        random_index = random.sample(range(0, X.shape[0]), self.n_clusters)
        self.centroids = X[random_index]

        for i in range(self.max_iter):
            # STEP 3: assign every point to its nearest centroid
            cluster_group = self.assign_clusters(X)
            old_centroids = self.centroids
            # STEP 4: move every centroid to the mean of its points
            self.centroids = self.move_centroids(X, cluster_group)
            # STEP 5: stop if the centroids did not move
            if np.array_equal(old_centroids, self.centroids):
                break

        return cluster_group

    def assign_clusters(self, X):
        cluster_group = []
        distances = []

        for row in X:
            for centroid in self.centroids:
                distances.append(np.sqrt(np.dot(row - centroid, row - centroid)))
            min_distance = min(distances)
            index_position = distances.index(min_distance)
            cluster_group.append(index_position)
            distances.clear()

        return np.array(cluster_group)

    def move_centroids(self, X, cluster_group):
        new_centroids = []

        cluster_type = np.unique(cluster_group)

        for type in cluster_type:
            new_centroids.append(X[cluster_group == type].mean(axis=0))

        return np.array(new_centroids)
```

5.10 Plot the result

In app.py we receive **y_means** and plot each cluster with boolean indexing, like in Part 4. With two clusters we write two scatter lines (red and blue):

app.py · plotting

```
plt.scatter(X[y_means == 0, 0], X[y_means == 0, 1], color="red")
plt.scatter(X[y_means == 1, 0], X[y_means == 1, 1], color="blue")
plt.show()
```

More than two clusters

Add a third centre such as (−2.5, 2.5) to **centers** (and a std value), set n_clusters = 3, and add a **third scatter line** (green, cluster 2). At first you may see wrong output simply because the plot code only drew two clusters. For four clusters add (2.5, −2.5) and a fourth scatter (yellow, cluster 3). Instead of writing a scatter per cluster, a loop is cleaner and works for any K:

app.py · loop version

```
colors = ["red", "blue", "green", "yellow", "purple", "orange"]
for k in range(km.n_clusters):
    plt.scatter(X[y_means == k, 0], X[y_means == k, 1], color=colors[k])
plt.scatter(km.centroids[:, 0], km.centroids[:, 1],
            marker="X", s=200, color="white", edgecolor="black")
plt.show()
```

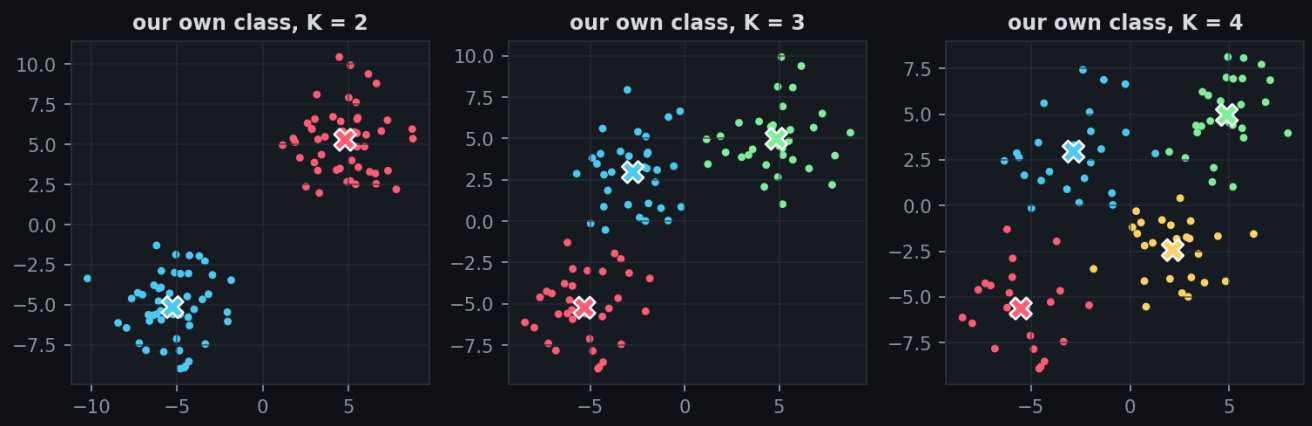


Figure 21: Our own class running with 2, 3 and 4 clusters.

5.11 Experiments: what does max_iter do?

Reduce **max_iter** and watch what happens. With max_iter = 10 the result is often still fine; with **max_iter = 1** the algorithm only does a single assign-and-move round, and if the random starting points were not well placed, the clusters are not formed properly. That is why max_iter should be generous (for example 100 or even 200): it is only an upper limit, since the loop already breaks when the centroids stop moving.

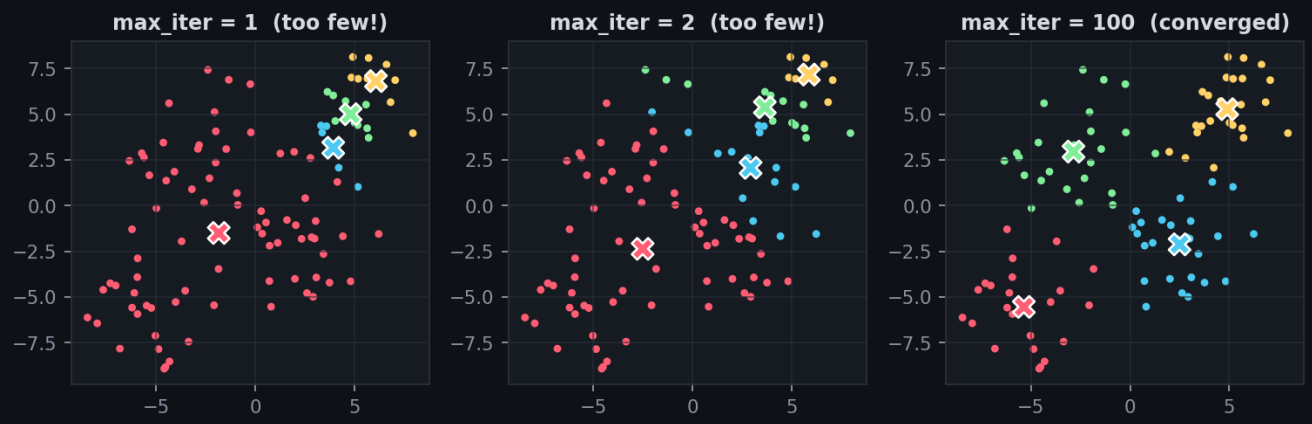


Figure 22: Same data, same random start. With 1 or 2 iterations the clusters are not formed yet; with 100 the algorithm converges.

Random start = random results

Our class picks random starting points, so two runs can give slightly different answers (and a rare unlucky start can give a worse one). Fix the seed with **random.seed(42)** for repeatable runs. Part 6 shows how to handle this properly.

5.12 Test on the real student dataset

Replace make_blobs with the student CSV: read it with **read_csv**, extract **X = df.values**, keep the same loop, set n_clusters = 4, maybe raise max_iter, and run. The output is exactly the picture we got from scikit-learn.

app.py · student data with our own class

```
import pandas as pd
df = pd.read_csv("student_clustering.csv")
X = df.values

km = KMeans(n_clusters=4, max_iter=100)
y_means = km.fit_predict(X)
```

Homework: implement the elbow method yourself

One thing is missing from our class: the elbow method. Implement it! You only need to calculate WCSS: for each cluster, sum the squared distances of its points to its centroid. Loop K = 1 … 10, fit your KMeans, store the WCSS and plot the curve. Try first, then compare with the solution below.

Solution · elbow from scratch (tested: bends at K = 4 on the student data)

```
def wcss(X, cluster_group, centroids):
    total = 0
    for k in range(len(centroids)):
        points = X[cluster_group == k]
        total += ((points - centroids[k]) ** 2).sum()
    return total

results = []
for k in range(1, 11):
    best = None
    for attempt in range(5):                  # a few random starts, keep the best
        km = KMeans(n_clusters=k, max_iter=100)
        g = km.fit_predict(X)
        if len(km.centroids) == k:            # skip runs that lost a cluster
            w = wcss(X, g, km.centroids)
            best = w if best is None else min(best, w)
    results.append(best)

plt.plot(range(1, 11), results, marker="o"); plt.show()
```

Check yourself (Part 5)

- Why does the formula with np.dot work in 3D, 10D, 100D? *(a − b automatically has one entry per dimension.)*
- What would happen without distances.clear()? *(The list grows, so index() returns wrong cluster numbers.)*
- Why axis=0 in mean? *(One mean per column: x-mean and y-mean.)*

LEVEL 6 · PRO

Part 6: Pro-Level K-Means

Objective function, k-means++, scaling, limits, silhouette, speed, real applications

Part 6: Pro-Level K-Means

You now know what K-Means does and how to code it. This part answers the questions professionals ask: *What is it really optimising? Why does it always stop? When does it fail? How do I make it fast, stable and trustworthy?*

6.1 What K-Means really optimises

K-Means is a search for the grouping with the smallest possible WCSS. Formally it tries to solve:

$$\min_{C_1 \dots C_K,\ \boldsymbol{\mu}_1 \dots \boldsymbol{\mu}_K} J = \sum_{j=1}^{K} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$

This is the same WCSS from Part 3, now called the **objective function J**. The algorithm we learned (also called **Lloyd's algorithm**) is an **alternating minimisation**:

Step	What is fixed	What we optimise	Why it never increases J
Assign (step 3)	Centroids	Cluster membership	Each point picks its nearest centroid, so its squared distance can only shrink or stay.
Move (step 4)	Membership	Centroid positions	For a fixed group of points, the **mean** is exactly the point that minimises the sum of squared distances.

Why the mean? (short proof)

For one cluster in 1D, define $f(c) = \Sigma\ (x_i - c)^2$. Differentiate: $f'(c) = -2\ \Sigma\ (x_i - c)$. Setting it to zero gives $\Sigma\ x_i = n \cdot c$, so $\mathbf{c = (\Sigma\ x_i)/n}$, the mean. The second derivative is $2n > 0$, so it is a minimum. The same argument works for each coordinate in n dimensions. This is why the centroids are *means*, and why the algorithm is named K-*Means*.

Because J never increases and there are only finitely many ways to split n points into K groups, the algorithm **must stop after a finite number of rounds** (Figure 10 showed WCSS going down at every step). **But** it only reaches a *local* minimum: a solution nothing nearby can improve, not necessarily the best of all. Finding the true global optimum is NP-hard, so we live with good local solutions.

6.2 Initialisation matters: n_init and k-means++

Since the first centroids are random, an unlucky start (for example two centroids inside one blob, none in another) can lock the algorithm into a poor local minimum.

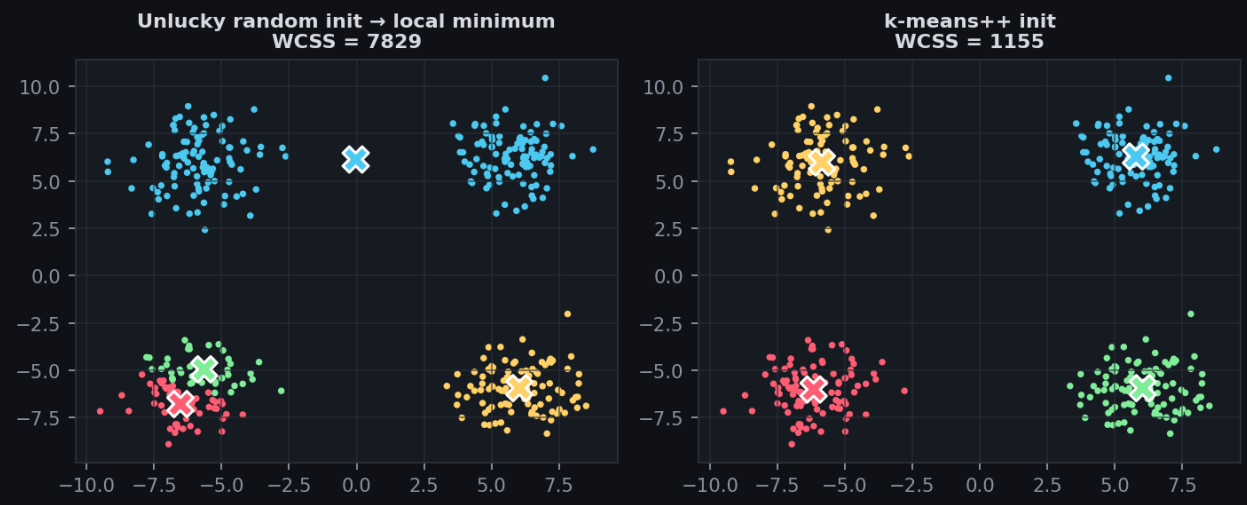


Figure 23: Same data. Left: an unlucky random start ends with WCSS = 7829. Right: k-means++ ends with WCSS = 1155.

Two professional remedies:

- **Multiple restarts (n_init):** run K-Means several times from different random starts and keep the run with the lowest WCSS. scikit-learn does this for you.
- **k-means++:** choose the starting centroids *smartly* so they are spread out. It is the default in scikit-learn.

k-means++ in 4 steps

1) Pick the first centroid uniformly at random from the data.
2) For every point x, compute D(x) = distance to the *nearest centroid already chosen*.
3) Choose the next centroid from the data with probability proportional to D(x)²:

$$P(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{x}'} D(\mathbf{x}')^2}$$

4) Repeat 2-3 until K centroids are chosen, then run normal K-Means. Far-away points get a high probability, so the starting centroids spread over the data.

6.3 Feature scaling is not optional

K-Means is built on Euclidean distance, so a feature with **big numbers** dominates the distance. Imagine clustering students by CGPA (0-10) and family income (20,000-80,000): the income differences are thousands of times larger, so the algorithm effectively ignores CGPA.

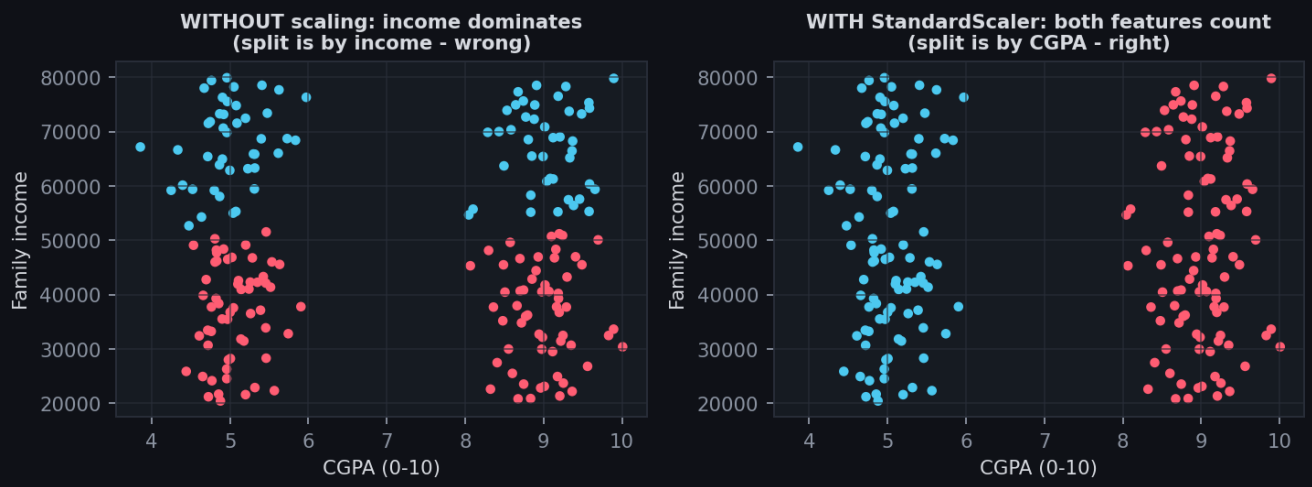


Figure 24: Without scaling the split follows income (agreement with the true groups: -0.00); after standardising it follows CGPA (agreement: 1.00).

Standardisation (z-score)

Rescale every feature to mean 0 and standard deviation 1:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the feature's mean and σ its standard deviation. After this all features speak with equal volume.

Python · scale first, then cluster

```python
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X_scaled = StandardScaler().fit_transform(X)
labels = KMeans(n_clusters=4, n_init=10, random_state=0).fit_predict(X_scaled)
```

Categorical columns

The transcript's example includes “which state is the student from”. K-Means cannot take the mean of a category. Options: one-hot encode (then scale), or use algorithms made for mixed data (K-Modes / K-Prototypes). In our toy data both columns were numeric and on a similar scale, which is why we did not need scaling there.

6.4 When K-Means fails (assumptions & limits)

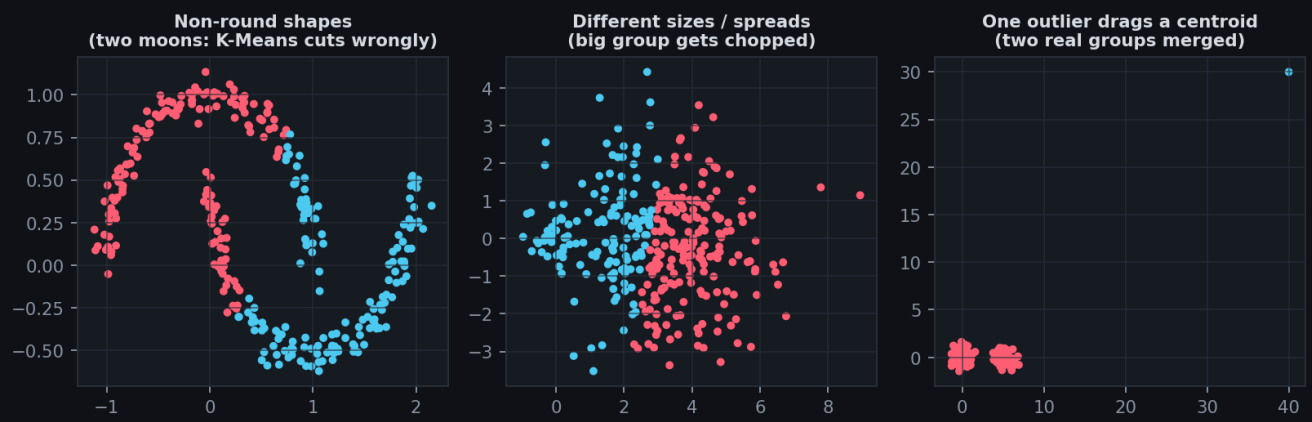


Figure 25: Three classic failure cases.

Problem	Why	Alternative / fix
Non-round clusters (moons, rings)	Nearest-centroid means boundaries are straight lines; clusters are assumed blob-shaped	DBSCAN, spectral clustering
Very different sizes or spreads	A big wide cluster looks like several to K-Means	Gaussian Mixture Models
Outliers	One extreme point pulls a mean, since means are sensitive	Remove outliers; K-Medoids
Must choose K	The algorithm cannot discover K	Elbow, silhouette, domain knowledge
High dimensions	Distances become similar ('curse of dimensionality')	PCA first, then K-Means
Empty clusters	A centroid can end up with no points	Re-seed that centroid (sklearn does this)

6.5 A second opinion on K: the silhouette score

The elbow can be ambiguous. The **silhouette score** measures how well each point fits *its own* cluster compared with the *next-closest* cluster:

Silhouette

For point i, let a(i) = average distance to the other points in its own cluster and b(i) = average distance to the points of the nearest other cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}} \qquad (-1 \le s \le 1)$$

Near **+1**: the point is well inside its cluster. Near **0**: on the border between two clusters. Near **−1**: probably in the wrong cluster. Average over all points and pick the K with the highest score.

Python · silhouette for each K

```python
from sklearn.metrics import silhouette_score

for k in range(2, 11):
    labels = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(X)
    print(k, round(silhouette_score(X, labels), 3))
```

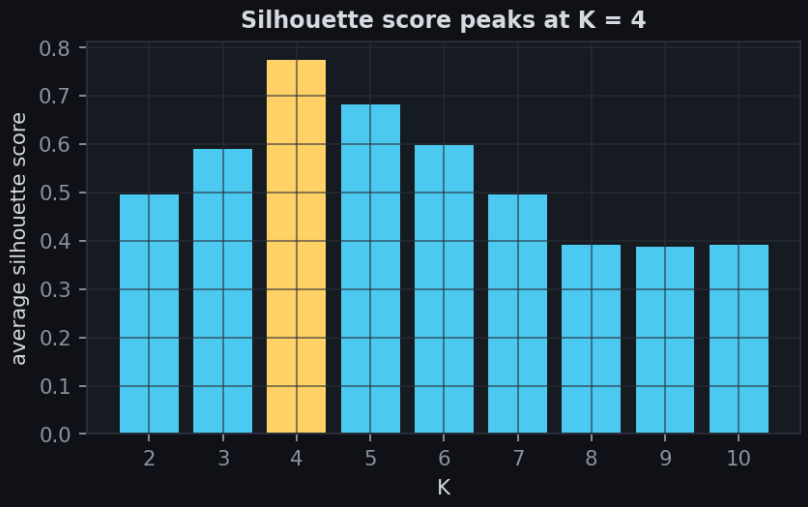


Figure 26: On the student data the silhouette also peaks at K = 4, agreeing with the elbow.

6.6 Speed: vectorise, and know the cost

Each round costs about:

$$O(n \cdot K \cdot d \cdot t)$$

where n = number of points, K = clusters, d = features, t = iterations: it is **linear** in the data size, which is why K-Means scales to big data. Our from-scratch loop, however, is slow because Python loops are slow. NumPy **broadcasting** computes all n × K distances at once:

kmeans.py · vectorised assign_clusters

```
def assign_clusters_fast(self, X):
    # X[:, None, :]  -> shape (n, 1, d)     self.centroids[None, :, :] -> (1, K, d)
    diff = X[:, None, :] - self.centroids[None, :, :]       # (n, K, d)
    distances = np.sqrt((diff ** 2).sum(axis=2))            # (n, K)
    return distances.argmin(axis=1)                         # nearest centroid index
```

On 3,000 points and 4 centroids we measured the loop version at about **28 ms** and the vectorised version at about **0.5 ms**, a speed-up of roughly **57×**, with identical output (verified). For huge datasets use **MiniBatchKMeans**, which updates centroids from small random batches.

6.7 A real application: image compression

“Images can be clustered” in a very literal way. Treat every pixel as a point with 3 features (red, green, blue). Run K-Means with K colours, then repaint every pixel with its centroid's colour. A photo with millions of colours becomes a picture with only K colours: this is **colour quantisation**.

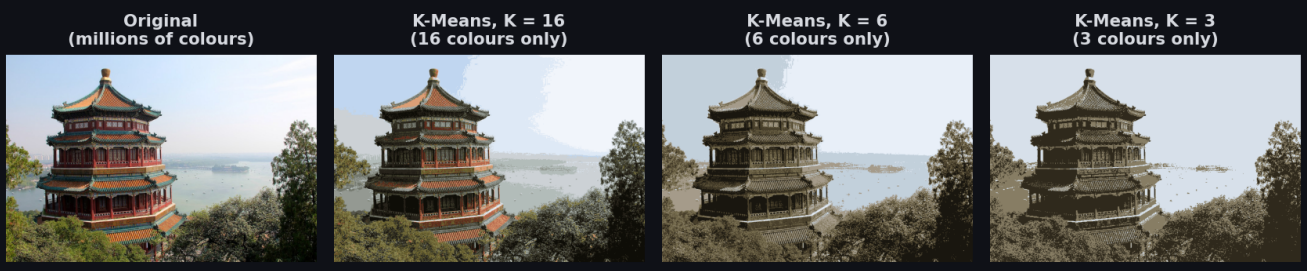


Figure 27: A photo reduced to 16, 6 and 3 colours by K-Means on pixel colours.

Python · colour quantisation

```python
from sklearn.datasets import load_sample_image
img = load_sample_image("china.jpg") / 255.0
h, w, _ = img.shape
pixels = img.reshape(-1, 3)                    # every pixel = a 3D point (R, G, B)

km = KMeans(n_clusters=16, n_init=3, random_state=0).fit(pixels[::50])
compressed = km.cluster_centers_[km.predict(pixels)].reshape(h, w, 3)
plt.imshow(compressed); plt.show()
```

6.8 scikit-learn KMeans: the parameters that matter

Parameter	Meaning	Pro advice
n_clusters	K	Choose with elbow + silhouette + domain sense.
init	'k-means++' (default) or 'random'	Keep the default.
n_init	How many restarts; best WCSS wins	10 is a safe habit (default 'auto' in new versions).
max_iter	Max rounds per run	Default 300 is rarely a limit.
tol	Stop when centroid movement is below this	Looser = faster, rougher.
random_state	Seed	Always set it for reproducible results.
algorithm	'lloyd' or 'elkan'	Elkan skips needless distance calculations (needs well-behaved data).

6.9 Pro checklist

- Scale numeric features (z-score) before K-Means unless they are naturally comparable.
- Handle outliers and encode categories before clustering.
- Use k-means++ and n_init > 1; fix random_state.
- Pick K with **two** methods (elbow + silhouette) and sanity-check with domain knowledge.
- Inspect the clusters (centroids, sizes, profiles); a cluster is only useful if you can explain it.
- For non-round clusters or heavy noise, switch algorithm instead of forcing K-Means.
- For millions of rows use MiniBatchKMeans; for >50 features consider PCA first.

Interview-style questions

Question	Short answer
Is K-Means supervised?	No: unsupervised.
Does K-Means always converge?	Yes, to a local (not necessarily global) minimum.
Why Euclidean distance?	It matches the squared-error objective whose minimiser is the mean.
Time complexity?	O(n·K·d·t).
What if the elbow is unclear?	Use silhouette, gap statistic, domain knowledge.
How does K-Means treat outliers?	Badly: they pull centroids. Remove or use K-Medoids.

Appendix A: One-page cheat sheet

Topic	Remember
Goal	Group similar points; K groups; each described by its centroid (mean).
5 steps	1) choose K 2) random centroids 3) assign to nearest 4) move to mean 5) stop if centroids unchanged, else repeat 3-5.
Distance	$d = \sqrt{\Sigma(a_i - b_i)^2}$ → np.sqrt(np.dot(a−b, a−b))
Centroid	μ = mean of the cluster's points → X[labels == k].mean(axis=0)
WCSS / inertia	Σ over clusters Σ over points $\|x - \mu\|^2$ → km.inertia_
Elbow method	Plot WCSS vs K (K = 1…10); choose the K where the steep fall flattens.
K = n	WCSS = 0 (useless); K = 1 gives the highest WCSS.
Pro	Scale features, k-means++, n_init, silhouette, watch outliers & shapes.

Appendix B: Glossary

Term	Meaning
Unsupervised learning	Learning from data without labels.
Cluster	A group of similar points.
Centroid	Mean point of a cluster.
Euclidean distance	Straight-line distance.
WCSS / inertia	Within-Cluster Sum of Squares, a measure of tightness.
Elbow point	K where the WCSS curve stops dropping steeply.
Convergence	State where nothing changes any more.
Local minimum	A solution that cannot be improved by small changes, but may not be the best overall.
k-means++	Smart, spread-out initialisation.
Silhouette	Score from −1 to 1 for how well points fit their clusters.
Standardisation	Rescaling features to mean 0, standard deviation 1.

Appendix C: Notes on the original scripts

The three source scripts were raw speech-to-text transcriptions of videos. While turning them into this tutorial, some garbled words were corrected from context, and the following were interpreted:

In the transcript	Interpreted as
“13 inch plus”	K-Means

In the transcript	Interpreted as
“factory name Khurram's screw makers”	“clustering algorithm”
“end-to-end geometry and intuition everywhere”	“geometric intuition”
“if you joke with me that, friends…”	“if you pay attention up to here…”
“you can request this” (songs)	songs can be clustered (for recommendation)
“promise you are falling from a mountain”	“imagine you are falling…” (mountain analogy)
Dry-run numbers (1.5, 4.5, 5.5…) in the script	Replaced by the correct values for X = [[1,2],[2,3],[3,4],[5,6]]: cluster means [2.5, 3.5] and [3, 4]

Additions beyond the scripts (nothing was removed): vocabulary table, numeric worked examples, maths derivations, diagrams, a runnable data generator, and all of Part 6.

The dataset in this tutorial is regenerated (scaled to a 0-10 range) so that the exact WCSS numbers differ from the video (which showed about 408 at K = 1); the shape of the curve and the elbow at K = 4 are the same.